Equipo #1	
Nombre	**Matrícula**
Sergio Chiw Wah Tam Hernández	1990036
Luis Fernando Montes de Oca Delgado	2001274
Gustavo Guzmán Puente	2060982
Nombre del curso: Analítica e Inteligencia de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Unidad: 4. Flujos de información y procesos de negocio	**Actividad:** Laboratorio ETL: Data Mart de ventas por estado y fecha
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Analítica e Inteligencia de Negocios. (2026). Unidad 4: Flujos de información y procesos de negocio [Manual de lectura y aprendizaje, versión para estudiantes]. Material del curso.

Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3.ª ed.). Wiley.

Oracle Corporation. (2026). MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/

Oracle Corporation. (2026). MySQL Workbench manual. https://dev.mysql.com/doc/workbench/en/

Objetivo:

Construir, con MySQL, un Data Mart de ventas por estado y fecha a partir de dos fuentes con errores intencionales (clientes.csv y pedidos.sql) de una tienda de comercio electrónico. El proceso debe aplicar las etapas de un flujo ETL (perfilado, staging, normalización, validación, carga incremental, conciliación e indicadores) sin modificar los datos originales, dejando trazabilidad de cada registro rechazado y controles que permitan reejecutar el proceso de forma reproducible.

Procedimiento:

El procedimiento se organizó en siete pasos que siguen la arquitectura vista en la unidad: los datos aterrizan sin cambios en tablas raw_*, se limpian en tablas stg_*, los registros inválidos se separan en rechazos_pedidos con su motivo y los datos válidos se cargan al modelo dimensional (dim_cliente y fact_ordenes), del que se obtienen los indicadores. Todo el proceso está en el script propio 03_etl_unidad4.sql, que se entrega junto con este reporte.

```
raw_*  (datos tal como llegaron)
   →  stg_*  (datos limpios y normalizados)
         ↘  rechazos_pedidos  (registros inválidos, con su motivo)
   →  dim_cliente / fact_ordenes  (modelo dimensional)
   →  indicadores  (ventas netas por estado y fecha)
```

Preparación del entorno y carga de los datos

Se instaló MySQL Server 8.4 LTS, que cumple el requisito de versión 8.0 o superior y es la versión con soporte vigente (la serie 8.0 terminó su soporte en abril de 2026). La versión actual de MySQL Workbench (26.10) ya no incluye el Table Data Import Wizard que pide la práctica, por lo que se instaló el Workbench clásico 8.0.47. Después se ejecutaron 01_creacion_bd.sql (base u4_etl y sus 9 tablas) y pedidos.sql.

Durante la carga se presentaron tres incidentes que muestran por qué es necesario verificar los datos después de cada carga y no confiar en que la herramienta funcionó:

- **pedidos.sql se ejecutó dos veces** y raw_pedidos quedó con 4,024 filas. Se vació con TRUNCATE (que también reinicia el contador de fila_id) y se volvió a ejecutar una sola vez.
- **Mapeo incorrecto en el asistente.** Los CSV traen al inicio una marca invisible de codificación (BOM), por lo que el asistente mostró la primera columna como "ï»¿id_cliente" y la asignó a fila_id. El resultado fue id_cliente vacío en todas las filas y 10 clientes perdidos, porque sus IDs repetidos chocaron con la llave primaria.
- **Acentos dañados.** Aun seleccionando UTF-8, el asistente guardó "Ciudad de México" como "Ciudad de MÃ©xico". Como solución se cargaron ambos CSV con el script 02_carga_clientes_catalogo.sql, que inserta los valores exactos de los archivos (SET NAMES utf8mb4) sin corregir nada. Los archivos originales no se modificaron.

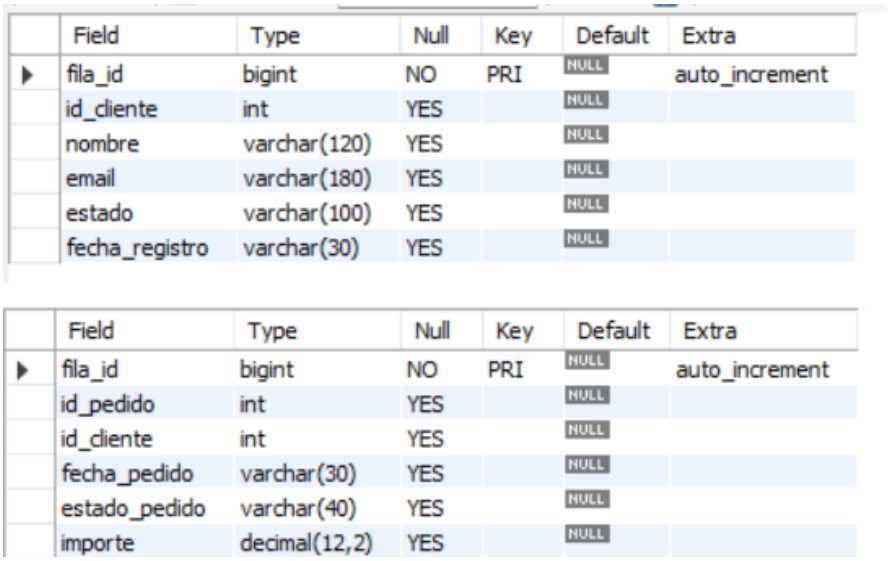
En catalogo_estados quedaron 16 de los 32 alias del archivo. La base usa la intercalación utf8mb4_unicode_ci, que no distingue mayúsculas ni acentos, así que para MySQL "nuevo león", "Nuevo León" y "NUEVO LEÓN" son el mismo valor de llave primaria y los repetidos se omitieron con INSERT IGNORE. El catálogo sigue funcionando porque las comparaciones contra él también ignoran mayúsculas y acentos. Con la carga corregida se obtuvieron 510 filas de clientes y 2,012 de pedidos (Figura 1).

	tabla	filas	ids_unicos
▶	clientes	510	500
	pedidos	2012	2000

Figura 1. *Conteo de filas e identificadores únicos en las tablas raw después de la carga.*

Paso 1. Perfilado

Antes de limpiar se midió la estructura y la calidad de los datos. En las tablas raw las fechas son VARCHAR (aceptan cualquier texto) y la llave primaria es fila_id, un consecutivo artificial. La zona de aterrizaje no debe rechazar nada, ni siquiera duplicados, para que los errores puedan medirse. Las claves de negocio son id_cliente e id_pedido, y raw_pedidos.id_cliente funciona como llave foránea lógica sin restricción declarada (Figura 2).



Field	Type	Null	Key	Default	Extra
fila_id	bigint	NO	PRI	NULL	auto_increment
id_cliente	int	YES		NULL	
nombre	varchar(120)	YES		NULL	
email	varchar(180)	YES		NULL	
estado	varchar(100)	YES		NULL	
fecha_registro	varchar(30)	YES		NULL	

Field	Type	Null	Key	Default	Extra
fila_id	bigint	NO	PRI	NULL	auto_increment
id_pedido	int	YES		NULL	
id_cliente	int	YES		NULL	
fecha_pedido	varchar(30)	YES		NULL	
estado_pedido	varchar(40)	YES		NULL	
importe	decimal(12,2)	YES		NULL	

Figura 2. *Estructura de raw_clientes y raw_pedidos (DESCRIBE).*

Tabla 1. *Hallazgos del perfilado.*

Tabla	Prueba	Resultado	Interpretación
raw_clientes	Filas / IDs únicos	510 / 500	10 IDs repetidos (1 al 10): copias exactas al final del archivo
raw_clientes	Variantes de escritura del estado	25	24 formas escritas (GTO, Guanajuato, GUANAJUATO…) más el vacío
raw_clientes	Estado vacío / sin equivalencia	13 / 13	Los únicos estados sin equivalencia en el catálogo son los vacíos
raw_clientes	Email vacío	9	Dato de contacto incompleto
raw_pedidos	Filas / IDs únicos	2,012 / 2,000	12 pedidos repetidos (1 al 12): riesgo de doble conteo
raw_pedidos	Formato de fecha	1,333 / 666 / 13	ISO AAAA-MM-DD / DD/MM/AAAA / vacía
raw_pedidos	Importe ≤ 0	37	Mínimo −5,571.81; ¿error o devolución?
raw_pedidos	Variantes de estatus	6	4 estatus reales escritos de distintas formas (entregado / ENTREGADO / Entregado)
raw_pedidos	Cliente inexistente	20	Todos con id_cliente 9999, aparentemente un valor de prueba o relleno

Un hallazgo importante fue que un GROUP BY normal esconde las variantes de escritura: por la intercalación de la base, "Guanajuato" (31) y "GUANAJUATO" (27) aparecen juntos como 58. Para ver las variantes reales se agrupó con COLLATE utf8mb4_bin, que compara byte por byte (Figuras 3 a 5).

	id_cliente	veces
▶	1	2
	2	2
	3	2
	4	2
	5	2
	6	2
	7	2
	8	2
	9	2
	10	2

id_pedido	veces
1	2
2	2
3	2
4	2
5	2
6	2
7	2
8	2
9	2
10	2
11	2
12	2

Figura 3. *Identificadores repetidos en clientes (izquierda) y pedidos (derecha).*

COAH	16
COAHUILA	17
Ciudad de México	24
Coahuila	16
GTO	19
GUANAJUATO	27
Guanajuato	31
JAL	21
JALISCO	18
Jalisco	20
NL	16
NUEVO LEÓN	22
Nuevo León	20
PUE	21
PUEBLA	22
Puebla	15
QRO	20
QUERÉTARO	25
Querétaro	19
TAMAULIPAS	19
TAMPS	21
Tamaulipas	20

	estatus_exacto	pedidos
▶	Cancelado	329
	ENTREGADO	349
	Entregado	343
	Enviado	303
	Pendiente	335
	entregado	353

	estado	clientes
▶		13

Figura 4. *Variantes exactas de estado y de estatus de pedido, y clientes sin equivalencia en el catálogo.*

formato	pedidos
DD/MM/AAAA	666
ISO (AAAA-MM-DD)	1333
vacía	13

importe_nulo	importe_no_positivo	minimo	maximo
0	37	-5571.81	5988.66

id_cliente	pedidos
9999	20

Figura 5. *Formatos de fecha, importes no positivos y pedidos de clientes inexistentes.*

Los resultados se guardaron en la tabla perfil_observaciones mediante un INSERT … SELECT con UNION ALL, donde cada SELECT calcula una prueba. Así el perfilado es reproducible: si llegan datos nuevos, se vuelve a ejecutar el bloque y la tabla se actualiza (Figura 6).

id	fecha_perfilado	tabla	columna	prueba	resultado	comentario
1	2026-10-28 20:42:49	raw_clientes	*	Total de filas	510.00	Filas recibidas de clientes.csv
2	2026-10-28 20:42:49	raw_clientes	id_cliente	IDs con más de una fila	10.00	Clave natural repetida
3	2026-10-28 20:42:49	raw_clientes	estado	Variantes de escritura (exactas)	25.00	Mayúsculas, acentos y abreviaturas distintas
4	2026-10-28 20:42:49	raw_clientes	estado	Estado vacío	13.00	Sin dato de ubicación
5	2026-10-28 20:42:49	raw_clientes	estado	Sin equivalencia en catálogo	13.00	No se puede asignar clave estatal
6	2026-10-28 20:42:49	raw_clientes	email	Email vacío	9.00	Dato de contacto incompleto
7	2026-10-28 20:42:49	raw_pedidos	*	Total de filas	2012.00	Filas cargadas por pedidos.sql
8	2026-10-28 20:42:49	raw_pedidos	id_pedido	IDs con más de una fila	12.00	Pedido duplicado: riesgo de doble conteo
9	2026-10-28 20:42:49	raw_pedidos	fecha_pedido	Fecha vacía	13.00	No se puede ubicar en el tiempo
10	2026-10-28 20:42:49	raw_pedidos	fecha_pedido	Formato ISO AAAA-MM-DD	1333.00	NULL
11	2026-10-28 20:42:49	raw_pedidos	fecha_pedido	Formato DD/MM/AAAA	666.00	Requiere conversión a DATE
12	2026-10-28 20:42:49	raw_pedidos	importe	Importe <= 0	37.00	¿Error o devolución? Pendiente de confirmar…
13	2026-10-28 20:42:49	raw_pedidos	estado_pedido	Variantes de escritura (exactas)	6.00	entregado / ENTREGADO / Entregado
14	2026-10-28 20:42:49	raw_pedidos	id_cliente	Cliente inexistente	20.00	Pedido sin cliente en raw_clientes
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figura 6. *Tabla perfil_observaciones con las 14 pruebas del perfilado.*

Paso 2. Staging

Se usaron dos zonas con propósitos distintos. Las tablas raw_* son la zona de aterrizaje inmutable: conservan los datos exactamente como llegaron, con errores, y nunca se modifican. Las tablas stg_* son la zona de datos normalizados: usan la clave de negocio como llave primaria (lo que impide duplicados), guardan las fechas como DATE y los estados como clave del catálogo, y se vacían y recalculan en cada ejecución.

No se debe modificar la información original por cuatro razones. Primero, la **trazabilidad**: si un indicador resulta extraño, se puede rastrear hasta el dato exacto que llegó. Segundo, la **reproducibilidad**: si una regla de limpieza resulta equivocada, se corrige la regla y se reprocesa desde raw, lo que sería imposible si el origen ya se hubiera alterado. Tercero, la **auditoría**: la conciliación del Paso 6 (recibidas = aceptadas + rechazadas) solo funciona si raw conserva el total original. Cuarto, la **responsabilidad**: los errores se corrigen en el sistema fuente, no en el almacén analítico.

Para comprobar que el proceso no altera el origen se obtuvo una huella con CHECKSUM TABLE antes de transformar (Figura 7), que se compara con la del Paso 6.

Table	Checksum
u4_etl.raw_clientes	618684399
u4_etl.raw_pedidos	2116921975
u4_etl.catalogo_estados	2323849249

Figura 7. *Huella (checksum) de las tablas de origen antes del ETL.*

Paso 3. Normalización

Clientes. stg_clientes se cargó uniendo cada cliente con el catálogo mediante LEFT JOIN, para que los clientes sin equivalencia no desaparecieran. Los duplicados se resolvieron con ROW_NUMBER(), conservando una fila por id_cliente según una regla de prioridad explícita: la última versión recibida (mayor fila_id). El email vacío se convirtió en NULL, porque un texto vacío y un dato desconocido no son lo mismo.

```
COALESCE(c.estado_canonico,
         CASE WHEN TRIM(COALESCE(r.estado,'')) = '' THEN 'NO_IDENT' ELSE 'SIN_CAT' END) AS
estado_canonico,
ROW_NUMBER() OVER (PARTITION BY r.id_cliente ORDER BY r.fila_id DESC) AS rn   -- se conserva rn
= 1
```

Estados sin equivalencia. Se distinguieron dos casos. NO_IDENT indica que el estado venía vacío (dato faltante). SIN_CAT indica que el estado venía escrito pero no existe en el catálogo; en ese caso el problema no es del cliente sino del catálogo, que debe actualizarse. En estos datos solo hubo casos NO_IDENT (13), pero la regla queda preparada para ambos. Se decidió conservar a estos clientes en lugar de eliminarlos: si se rechazaran, se perdería también la información de sus compras (Figura 8).

clientes_stg
500

estado_canonico	clientes
GTO	74
CDMX	70
QRO	63
TAMPS	59
PUE	58
JAL	57
NL	57
COAH	49
NO_IDENT	13

Figura 8. *stg_clientes: 500 clientes únicos distribuidos por clave estatal.*

Pedidos. La normalización de pedidos se dejó en una vista (v_pedidos_norm), porque stg_pedidos solo debe recibir pedidos válidos y la validación corresponde al Paso 4. La vista convierte las fechas a DATE según su formato, detectado con REGEXP, y unifica el estatus con UPPER(TRIM()). Las fechas que no corresponden a ningún formato quedan

como NULL en lugar de inventar un valor, y se conservan fila_id y la fecha original para mantener la trazabilidad (Figura 9).

```
CASE WHEN fecha_pedido REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(fecha_pedido,
'%Y-%m-%d')
     WHEN fecha_pedido REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(fecha_pedido,
'%d/%m/%Y')
     ELSE NULL END AS fecha_pedido
```

	fecha_original	fecha_pedido
▶	18/08/2026	2026-08-18
	04/03/2026	2026-03-04
	22/04/2026	2026-04-22
	11/07/2026	2026-07-11
	21/02/2026	2026-02-21

	estado_pedido	pedidos
▶	ENTREGADO	1045
	CANCELADO	329
	PENDIENTE	335
	ENVIADO	303

	sin_fecha_valida
▶	13

Figura 9. *Conversión de fechas DD/MM/AAAA, estatus unificados y pedidos sin fecha válida.*

Paso 4. Validación y rechazos

Cada fila de la vista normalizada se evaluó contra las reglas de la Tabla 2. Si falla alguna, se envía a rechazos_pedidos con su fila_id original y un motivo que incluye el valor que causó el rechazo; si pasa todas, se carga en stg_pedidos. Cada regla es un CASE que devuelve un texto cuando falla y NULL cuando se cumple. CONCAT_WS une los textos omitiendo los NULL, por lo que una fila que falla varias reglas registra todos sus motivos.

Tabla 2. *Reglas de validación de pedidos.*

Regla	Se rechaza si…
Pedido duplicado	El id_pedido ya apareció antes; se conserva la primera fila recibida (ROW_NUMBER por id_pedido)
Cliente inexistente	El id_cliente no existe en stg_clientes
Fecha inválida o vacía	La fecha no pudo convertirse a DATE
Importe no positivo	El importe es nulo o menor o igual a cero
Estatus no reconocido	No es ENTREGADO, CANCELADO, ENVIADO ni PENDIENTE
Estado del cliente no reconocido	El cliente tiene estado NO_IDENT o SIN_CAT

La actividad pide verificar que el pedido tenga un "estado reconocido". Se interpretó como el estatus del pedido y, además, se agregó la última regla: los pedidos de clientes sin estado se rechazan porque no pueden asignarse a ningún estado dentro de un Data Mart de ventas por estado. El cliente se conserva en la dimensión y sus pedidos quedan en cuarentena; cuando el estado se corrija en el sistema fuente, la siguiente ejecución los aceptará automáticamente. Esta regla rechaza 47 pedidos adicionales con un importe de $125,199.05.

El resultado fue 2,012 filas = 1,883 aceptadas + 129 rechazadas. Tres filas fallaron dos reglas a la vez (Figuras 10 y 11).

filas_raw	aceptadas	rechazadas
2012	1883	129

tipo_rechazo	filas
Estado del cliente no reconocido	47
Importe no positivo	37
Cliente inexistente	20
Fecha inválida o vacía	13
Pedido duplicado	12

Figura 10. *Filas aceptadas y rechazadas, y rechazos por tipo.*

fila_id	id_pedido	motivo
745	745	Fecha inválida o vacía (''); Estado del cliente no reconocido (NO_IDENT)
1068	1068	Importe no positivo (-5020.63); Estado del cliente no reconocido (NO_IDENT)
2011	11	Duplicado de id_pedido 11; Estado del cliente no reconocido (NO_IDENT)
NULL	NULL	NULL

Figura 11. *Filas rechazadas con más de un motivo.*

Paso 5. Carga incremental

dim_cliente y fact_ordenes se cargaron con UPSERT (INSERT … ON DUPLICATE KEY UPDATE) sobre las claves naturales id_cliente e id_pedido: si la clave no existe se inserta, y si ya existe se actualiza con los valores más recientes. A diferencia de las tablas de staging, estas tablas nunca se vacían. La dimensión se maneja como SCD Tipo 1 (se sobrescribe con el valor actual).

```
INSERT INTO fact_ordenes (id_pedido, id_cliente, fecha_pedido, estado_pedido, importe,
ultima_carga)
SELECT s.id_pedido, s.id_cliente, s.fecha_pedido, s.estado_pedido, s.importe, NOW() FROM
stg_pedidos s
ON DUPLICATE KEY UPDATE id_cliente = s.id_cliente, fecha_pedido = s.fecha_pedido,
  estado_pedido = s.estado_pedido, importe = s.importe, ultima_carga = NOW();
```

Para probar la idempotencia, el proceso se ejecutó tres veces sin agregar filas al origen (las pruebas 2 y 3 incluyeron los pasos 3 a 5 completos). ROW_COUNT() suma 1 por cada fila insertada, 2 por cada fila actualizada y 0 por cada fila sin cambios. En la primera ejecución los 1,883 pedidos se insertaron; en las siguientes los 1,883 se actualizaron (1,883 × 2 = 3,766) porque ultima_carga cambia, y ninguno se insertó. En dim_cliente el contador fue 0 porque nombre y estado no cambiaron. El número de hechos se mantuvo estable en 1,883 (Tabla 3 y Figura 12).

Tabla 3. *Prueba de idempotencia de la carga.*

Ejecución	Afectadas dim	Afectadas fact	Filas dim	Filas fact
Prueba 1	500	1,883	500	1,883
Prueba 2	0	3,766	500	1,883
Prueba 3	0	3,766	500	1,883

Prueba 1

filas_afectadas_dim
500

filas_afectadas_fact
1883

filas_dim	filas_fact	ultima_carga
500	1883	2026-10-28 21:10:15

Prueba 2

filas_afectadas_dim
0

filas_afectadas_fact
3766

filas_dim	filas_fact	ultima_carga
500	1883	2026-10-28 21:12:14

Prueba 3

filas_afectadas_dim
0

filas_afectadas_fact
3766

filas_dim	filas_fact	ultima_carga
500	1883	2026-10-28 21:13:23

Figura 12. *Resultados de las tres ejecuciones de la carga.*

Una clave estable permite la idempotencia porque el mismo pedido siempre llega con el mismo id_pedido desde el sistema fuente. Cuando el UPSERT encuentra un identificador ya cargado, actualiza en lugar de insertar, así que ejecutar la carga una o diez veces deja el mismo resultado. Si la tabla usara un identificador autoincremental como llave, cada ejecución generaría filas nuevas para los mismos pedidos y las ventas se duplicarían. Una limitación es que el UPSERT no borra: si un pedido ya cargado se volviera inválido en una ejecución futura, seguiría en fact_ordenes y se necesitaría una regla adicional para detectarlo.

Paso 6. Conciliación

Se verificó que no se perdiera ni se inventara información en el proceso. En filas, cada registro de raw terminó en staging o en rechazos, sin quedar en ambos ni en ninguno: 2,012 − 1,883 − 129 = 0. En importes, el total aceptado en staging coincide con el de la tabla de hechos ($5,805,137.45) y la diferencia contra el total bruto es exactamente el importe rechazado. El importe rechazado se obtuvo uniendo rechazos_pedidos con raw_pedidos por fila_id, que es la trazabilidad en uso (Figura 13).

filas_raw	aceptadas_unicas	rechazadas	diferencia
2012	1883	129	0

total_bruto	total_valido_stg	total_fact	total_rechazado
6023923.87	5805137.45	5805137.45	218786.42

tipo_rechazo	filas	importe
Estado del cliente no reconocido	47	125199.05
Cliente inexistente	20	71047.87
Fecha inválida o vacía	13	49402.13
Pedido duplicado	12	29836.84
Importe no positivo	37	-56699.47

Figura 13. *Cuadre de filas, cuadre de importes e importe rechazado por tipo.*

Tabla 4. *Conciliación de importes.*

Concepto	Importe
Total bruto (raw_pedidos)	$6,023,923.87
(−) Estado del cliente no reconocido (47)	$125,199.05
(−) Cliente inexistente (20)	$71,047.87
(−) Fecha inválida o vacía (13)	$49,402.13
(−) Pedido duplicado (12)	$29,836.84
(−) Importe no positivo (37)	−$56,699.47
= Total válido (stg_pedidos = fact_ordenes)	$5,805,137.45

El total bruto difiere del válido porque incluye importes que no representan ventas atribuibles: pedidos contados dos veces, pedidos de un cliente que no existe, pedidos sin fecha y pedidos sin estado. Además, los errores no siempre inflan las cifras: los 37 importes negativos reducen el total bruto en $56,699.47, así que un error también puede esconder ventas. El total válido todavía no equivale a ventas, porque incluye pedidos cancelados, pendientes y enviados.

La ejecución se registró en etl_auditoria con los conteos, el importe válido y una nota calculada que indica "OK" cuando todo cuadra (Figura 14). Finalmente, el checksum de las tablas de origen resultó idéntico al del Paso 2, lo que demuestra que el proceso no modificó los datos originales (Figura 15).

id	fecha_ejecucion	filas_origen	filas_validas	filas_rechazadas	filas_fact	ventas_validas	nota
1	2026-10-28 21:17:19	2012	1883	129	1883	5805137.45	OK: origen = válidas + rechazadas; importe stg = fact
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figura 14. *Registro de la ejecución en etl_auditoria.*

Table	Checksum
u4_etl.raw_clientes	618684399
u4_etl.raw_pedidos	2116921975
u4_etl.catalogo_estados	2323849249

Figura 15. *Checksum de las tablas de origen al final del ETL (idéntico al de la Figura 7).*

Paso 7. Indicadores

Supuesto documentado: para este laboratorio, las ventas netas se definen como la suma de importes de los pedidos con estatus ENTREGADO. No se modelan descuentos, impuestos ni devoluciones, por lo que esta cifra no equivale a las ventas contables reales. Los pedidos cancelados, pendientes y enviados no cuentan como venta efectiva.

El indicador se guardó como la vista v_kpi_ventas_estado_fecha, que une fact_ordenes con dim_cliente para obtener el estado de cada pedido (el hecho aporta la medida y la dimensión el contexto). El catálogo se redujo a una fila por estado antes de unirlo, porque tiene varios alias por estado y unirlo directamente multiplicaría cada pedido y las ventas.

Los pedidos se cuentan con COUNT(DISTINCT id_pedido) para que el indicador siga siendo correcto aunque la granularidad de la tabla de hechos cambie (Figura 16).

estado_canonico	nombre_oficial	fecha_pedido	pedidos_entregados	ventas_netas
CDMX	Ciudad de México	2026-01-03	2	4559.22
CDMX	Ciudad de México	2026-01-05	1	5552.05
CDMX	Ciudad de México	2026-01-07	1	3919.30
CDMX	Ciudad de México	2026-01-11	1	2798.42
CDMX	Ciudad de México	2026-01-12	2	10943.00
CDMX	Ciudad de México	2026-01-13	1	1309.99
CDMX	Ciudad de México	2026-01-14	3	9666.62
CDMX	Ciudad de México	2026-01-15	1	4663.87
CDMX	Ciudad de México	2026-01-19	2	8414.83
CDMX	Ciudad de México	2026-01-23	1	4555.86
CDMX	Ciudad de México	2026-01-29	1	618.34
CDMX	Ciudad de México	2026-01-30	1	609.51
CDMX	Ciudad de México	2026-02-02	1	2608.03
CDMX	Ciudad de México	2026-02-05	2	6829.00
CDMX	Ciudad de México	2026-02-09	1	1884.80
CDMX	Ciudad de México	2026-02-10	1	1898.22
CDMX	Ciudad de México	2026-02-14	2	9872.91
CDMX	Ciudad de México	2026-02-17	1	3991.77
CDMX	Ciudad de México	2026-02-19	1	4890.22
CDMX	Ciudad de México	2026-02-27	1	4668.37
CDMX	Ciudad de México	2026-02-28	1	4562.83
CDMX	Ciudad de México	2026-03-05	1	3447.02
CDMX	Ciudad de México	2026-03-07	3	9416.48
CDMX	Ciudad de México	2026-03-08	1	4905.32
CDMX	Ciudad de México	2026-03-09	1	1944.81
CDMX	Ciudad de México	2026-03-11	1	2809.94

v_kpi_ventas_estado_fecha58 × Result 59 Result 60 Result 61

Figura 16. *Consulta final: ventas netas y pedidos entregados por estado y fecha (extracto).*

Tabla 5. *Ventas netas y pedidos entregados por estado.*

Estado	Pedidos entregados	Ventas netas	% ventas
Guanajuato (GTO)	147	$429,540.88	14.51
Querétaro (QRO)	139	$421,205.17	14.23
Tamaulipas (TAMPS)	128	$411,303.28	13.89
Ciudad de México (CDMX)	136	$402,581.05	13.60
Puebla (PUE)	121	$380,517.27	12.85
Nuevo León (NL)	108	$342,629.61	11.57
Jalisco (JAL)	112	$316,748.84	10.70
Coahuila (COAH)	84	$256,042.92	8.65
Total	975	$2,960,569.02	100.00

estado_canonico	nombre_oficial	pedidos_entregados	ventas_netas	pct_ven
GTO	Guanajuato	147	429540.88	14.51
QRO	Querétaro	139	421205.17	14.23
TAMPS	Tamaulipas	128	411303.28	13.89
CDMX	Ciudad de México	136	402581.05	13.60
PUE	Puebla	121	380517.27	12.85
NL	Nuevo León	108	342629.61	11.57
JAL	Jalisco	112	316748.84	10.70
COAH	Coahuila	84	256042.92	8.65

Figura 17. *Resumen por estado obtenido de la vista de indicadores.*

estado_canonico	mes	pedidos_entregados	ventas_netas
CDMX	2026-01	17	57611.01
CDMX	2026-02	11	41206.15
CDMX	2026-03	18	45675.00
CDMX	2026-04	13	39139.45
CDMX	2026-05	18	55147.34
CDMX	2026-06	19	57593.71
CDMX	2026-07	20	55814.58
CDMX	2026-08	11	32533.15
CDMX	2026-09	9	17860.66
COAH	2026-01	11	41856.28
COAH	2026-02	11	36234.52
COAH	2026-03	9	27083.21
COAH	2026-04	9	26561.30
COAH	2026-05	10	26366.33
COAH	2026-06	7	22533.15
COAH	2026-07	7	19838.64
COAH	2026-08	12	25988.11
COAH	2026-09	8	29581.38
GTO	2026-01	25	77085.48
GTO	2026-02	16	43319.77
GTO	2026-03	18	41492.77
GTO	2026-04	15	37334.82
GTO	2026-05	18	58714.56
GTO	2026-06	12	34559.82
GTO	2026-07	18	54332.77
GTO	2026-08	13	49543.73

v_kpi_ventas_estado_fecha58 Result 59 Result 60 Result 61

Figura 18. *Tendencia mensual de ventas netas por estado (extracto).*

Como control final, los 975 pedidos entregados y los $2,960,569.02 de ventas netas del indicador coinciden con los calculados directamente sobre fact_ordenes (Figura 19). Guanajuato, Querétaro y Tamaulipas concentran las mayores ventas y Coahuila las menores. Los pedidos de clientes sin estado identificado no aparecen en el indicador porque fueron rechazados en el Paso 4.

pedidos_kpi	pedidos_fact	ventas_kpi	ventas_fact
975	975	2960569.02	2960569.02

Figura 19. *Control: el indicador cuadra con la tabla de hechos.*

Reflexión sobre las decisiones de limpieza

¿Qué ocurre si un cliente tiene estado vacío? ¿Se rechaza o se agrupa como no identificado? El cliente no se rechaza: se conserva en la dimensión con la clave NO_IDENT, porque es un cliente real y eliminarlo borraría también su historial. Sin embargo, como el Data Mart analiza ventas por estado, sus pedidos se envían a rechazos con el motivo "Estado del cliente no reconocido" y quedan en cuarentena hasta que el dato se corrija en el sistema fuente. Con esta decisión, el total válido es menor que el real en $125,199.05, y esa diferencia queda documentada en la conciliación en lugar de esconderse. También se distingue NO_IDENT (dato faltante) de SIN_CAT (estado escrito pero ausente del catálogo, que indica que el catálogo debe actualizarse).

¿Cómo detectarías pedidos que llegaron tarde? La pregunta admite dos interpretaciones. Si se refiere a entregas tardías, inicialmente consideré agregar un estatus "Entregado (Tarde)". Sin embargo, eso mezcla dos conceptos distintos (la etapa del pedido y su puntualidad) y haría que esos pedidos salieran del indicador de ventas y fueran rechazados como estatus no reconocido. Es mejor conservar ENTREGADO y agregar una columna aparte, como entrega_tardia o dias_retraso. Para calcularla faltan datos en las fuentes: la fecha real de entrega y la fecha prometida o plazo de entrega, que tendría que proporcionar el ERP (por ejemplo, entrega_tardia = fecha_entrega_real > fecha_compromiso).

Si se refiere a datos que llegan tarde al Data Mart (por ejemplo, un pedido de marzo que aparece hasta la carga de abril), se detectaría agregando a fact_ordenes una columna primera_carga que solo se llene al insertar, ya que ultima_carga se sobrescribe en cada UPSERT. Comparando primera_carga con las fechas de ejecución registradas en etl_auditoria se identifican los pedidos que debieron entrar en una carga anterior, lo que indica que los reportes ya publicados de ese periodo estaban incompletos.

¿Cómo modificarías el proceso para admitir devoluciones? Los importes negativos no deberían rechazarse automáticamente, porque pueden ser devoluciones legítimas. Propongo validar que exista una compra del mismo cliente que respalde la devolución y que lo devuelto sea menor o igual a lo comprado. Para que la regla sea exacta, la fuente debería indicar el pedido original (id_pedido_origen), y entonces se verificaría que ese pedido exista, pertenezca al mismo cliente, esté ENTREGADO, sea anterior a la devolución y que la suma de todas sus devoluciones no supere el importe original. Sin ese campo, buscar una compra del cliente con importe suficiente es solo una aproximación.

Las devoluciones válidas se registrarían por separado (una columna tipo_movimiento o una tabla fact_devoluciones) y las ventas netas pasarían a ser las ventas entregadas menos las devoluciones válidas, lo que eliminaría el supuesto del Paso 7. También habría que decidir si la devolución se resta en la fecha en que ocurre o en la de la venta original,

y documentarlo. Los negativos sin compra que los respalde seguirían rechazándose con un motivo como "Devolución sin compra que la respalde".

Otras decisiones de limpieza. Los duplicados se resolvieron con una regla de prioridad explícita (en clientes, la última versión recibida; en pedidos, la primera llegada), en lugar de un GROUP BY que ocultaría diferencias. No se inventaron fechas ni se convirtieron datos faltantes en cero: lo desconocido se guardó como NULL o se rechazó con su motivo. Los 37 importes negativos se rechazaron según la regla del laboratorio, pero se documentaron como posibles devoluciones pendientes de confirmar con el área de negocio.

Conclusiones

El laboratorio mostró que la calidad de los datos determina directamente los indicadores: de $6,023,923.87 brutos se pasó a $5,805,137.45 válidos y finalmente a $2,960,569.02 de ventas netas. Un tablero construido sobre los datos crudos habría contado pedidos duplicados, ventas de un cliente que no existe e importes negativos, y habría dado una imagen equivocada del negocio aunque su diseño fuera correcto.

También quedó claro que un proceso ETL no consiste solo en mover datos, sino en tomar y documentar decisiones de negocio: qué hacer con un estado vacío, cómo interpretar un importe negativo o qué cuenta como venta. Separar las zonas raw, staging y modelo dimensional, guardar cada rechazo con su motivo y cargar con claves estables hizo posible reejecutar el proceso sin duplicar datos y comprobar, con la conciliación y el checksum, que no se perdió ni se alteró información.

Por último, los problemas de carga con el asistente de importación (columnas mal asignadas y acentos dañados) demostraron en la práctica lo que plantea la unidad: no basta con que una herramienta termine sin errores. Hay que medir y verificar los datos después de cada etapa.